输入/输出(I/O)：在主存和外部设备(磁盘、终端、网络、键盘...)之间进行复制数据的过程

```
应用层                          ->软件
操作系统                         ->软件
处理器 + 主存 + IO设备             -> 硬件
```

不带操作系统和带操作系统的区别

```
不带操作系统   简单的，应用直接操作硬件
               应用开发的人必须要了解硬件的实现细节
               不能同时运行多个程序(无并发)
带操作系统     应用开发的人只要集中精力实现应用的开发逻辑
               不需要关心硬件的实现细节
               同时运行多个程序(无并发)
```

操作系统的功能：

```
提供很多的API接口
提供并发
管理和分配资源的系统软件
```

linux I/O

```
文件：
1)文件属性
    文件名 文件大小 文件类型 ...
    inode 唯一标识文件存在与否
2)文件内容

物理inode存在于磁盘中
当把u盘插入到电脑的那一刻，就会在系统中生成对应的逻辑inode
操作系统识别到一个文件(物理inode)，就会在操作系统中创建一个struct inode逻辑inode来描述文件的物理信息
    一个物理inode 对应 一个struct inode
    struct inode
    {
        文件名 文件大小 文件类型 ...
    }
```

```
linux下一切皆文件
系统中所有的输入输出都是通过系统IO函数调用来实现文件的读写，使得输入和输出操作得到同一
- 打开文件
    内核会返回一个非负整数(文件描述符fd)
    后续对这个文件的读写操作都需要通过拿到的文件描述符
    进程自动打开三个文件，标准输入文件(0) 标准输出文件(1) 标准出错文件(2)
- 关闭文件

- 读写文件
    eg:
        读取文件3个字节 abc
```

```
        读取文件3个字节 def

        "abcdefghadgddsdf"
- 光标的跳转
    lseek
```

内核打开文件的过程

```
内核用三个相关的结构体来描述打开的文件
- 文件描述符表(进程文件表项)     数组
    每个进程都会有一个进程文件表项
- 文件表(file table)
    用来描述一个打开的文件
    struct file
    {
        文件状态标记    可读/可写
        文件偏移量(光标)
        记录指向当前文件表的描述符的个数
        .....
        struct inode *   //指向逻辑上inode指针
    }
- 逻辑inode表
    struct inode
    {
        文件名 文件大小 文件类型 ...
    }
    硬链接
    1物理inode -> 多个逻辑inode
```

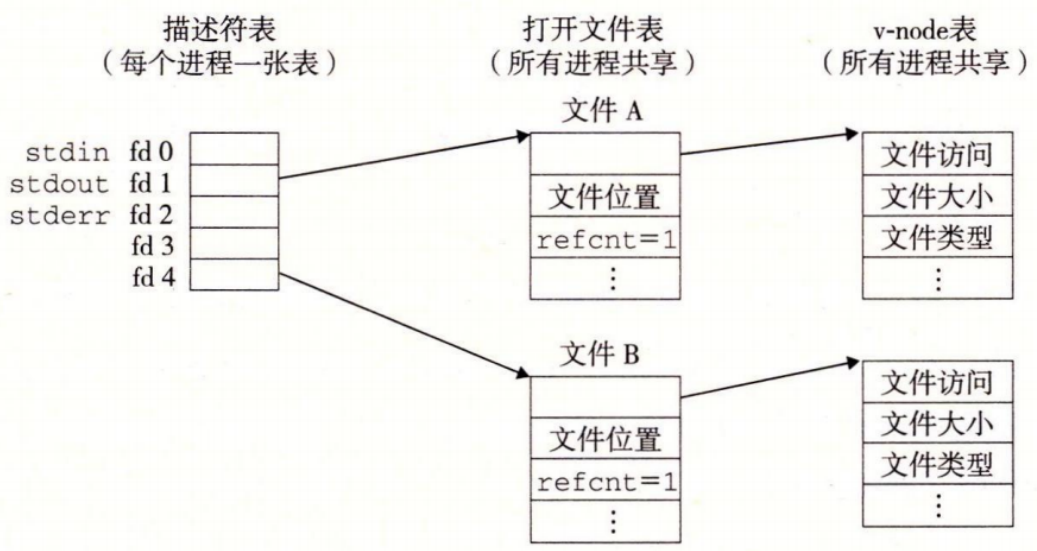

图 10-12　典型的打开文件的内核数据结构。在这个示例中，两个描述符引用不同的文件。没有共享

1.linux文件操作的接口

man 2 函数名

```
china@ubuntu:build$ man open
china@ubuntu:build$ man -f open
open (1)             - start a program on a new virtual terminal (VT).
open (2)             - open and possibly create a file
china@ubuntu:build$ man 2 open
china@ubuntu:build$
```

1.open

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int flags);//打开一个已经存在的文件
int open(const char *pathname, int flags, mode_t mode);//创建一个新的文件并打开
参数：
    pathname：要打开的文件或创建文件的路径(绝对路径或相对路径)
    flags：打开文件标记
        O_RDONLY只读  0
        O_WRONLY只写  1
        O_RDWR可读可写  2
        注意：上面的三个状态标记只能选择其中的一个
        O_CREAT文件如果不存在，则创建一个文件
        O_TRUNC文件已经存在，则进行截短(变成空的文件)
        O_APPEND追加，设置文件的偏移量会在文件末尾(写)
        O_NONBLOCK以非阻塞的方式打开文件，默认是阻塞
    eg:以只读的方式打开已经存在的文件1.txt
        int fd = open("1.txt",O_RDONLY);
    多个状态标记需要用到位或连接 |
    eg:打开一个已经存在的文件1.txt，并且以读写的方式打开，在文件的末尾写入数据
        int fd = open("1.txt",O_RDWR|O_APPEND);
    mode:指定创建文件的权限，当第二个参数有O_CREAT才会有第三个参数
        创建新的文件的权限 = mode & ~mask
        有两种指定权限的方式
        方式1：八进制
            0777
        方式2：宏

返回值：
    成功：返回一个打开文件的文件描述符 >2
    失败：-1，同时设置错误码
```

mode为宏

掩码	描述
S_IRUSR	使用者（拥有者）能够读这个文件
S_IWUSR	使用者（拥有者）能够写这个文件
S_IXUSR	使用者（拥有者）能够执行这个文件
S_IRGRP	拥有者所在组的成员能够读这个文件
S_IWGRP	拥有者所在组的成员能够写这个文件
S_IXGRP	拥有者所在组的成员能够执行这个文件
S_IROTH	其他人（任何人）能够读这个文件
S_IWOTH	其他人（任何人）能够写这个文件
S_IXOTH	其他人（任何人）能够执行这个文件

perror

```
#include <stdio.h>

void perror(const char *s);
功能：打印s字符串并再在后面加上错误的原因，会自动换行
eg：
    perror("");//这句话在打开失败的时候打印错误原因，并且会自动换行
```

练习：打开上一层目录的1.txt文件，如果失败请用perror打印错误信息，并且解决错误，直到成功为止，如果成功，打印ok

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int main()
{
    //打开
    int fd = open("../1.txt",2);
    if(fd == -1)//失败
    {
        perror("open failed");
    }
    else
    {
        printf("ok\n");
    }
}
```

umask

```
#include <sys/types.h>
#include <sys/stat.h>

mode_t umask(mode_t mask);
mask:指定文件的掩码
返回值：返回上一次的掩码

注意：
    在共享目录下去设置umask,最后的新文件的权限可能会与预期不符
    需要把运行的文件放到linux文件夹下运行 /home/china
```

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int main()
{
    // 求上一次的umask
    // int pre_umask_value = umask(0); // 2
    // printf("pre_umask_value:%o\n", pre_umask_value);

    // 创建一个新的文件
    int fd = open("./2.txt", 2 | O_CREAT, 0666);
    if (fd == -1)
    {
        perror("open fail");
        return 0;
    }

    printf("ok\n");

}
```

2.write

```
#include <unistd.h>

ssize_t  write(int fd, const void *buf, size_t count);
功能：把buf所指向的空间中count个字节 写入到fd所代表的文件中
参数：
    fd:写入的那个文件的描述符
    buf:指针，指向的空间用来保存写入到文件中的数据
    count:你想要写入多少个字节到文件中
返回值：
    成功：返回值实际要写入的字节数(<=count)
    失败：-1，errno被设置
```

往"1.txt"写入“abcd”

```
//1.打开1.txt
int fd = open("1.txt",....);
//2.写入
write(fd,"abcd",4);

char *buf = "abcdasdadasdhsdhashdad";
write(fd,buf,strlen(buf));

char buf[100] = "abcd";
int ret = write(fd,buf,strlen(buf));
prtintf("成功写入%d个字节\n",ret);
```

练习：使用开发板，把开发板显示成红色

```
1.打开屏幕  "/dev/fb0"
2.显示红色
    rk3568:1024*600
    6818:800*480

    三基色：红 绿 蓝
    一个颜色的描述(透明度 红 绿 蓝)，每个分量占1个字节(unsigned char)
    红        绿        蓝
    0xff     0x00      0x00        深红色0xff0000

    unsigned int color = 0xff0000;
    write(fd,&color,4);//循环1024*600次
3.关闭屏幕
    close(fd);
```

RGB颜色对照表

#FFFFFF	#FFFFF0	#FFFFE0	#FFFF00
#FFFAFA	#FFFAF0	#FFFACD	#FFF8DC
#FFF68F	#FFF5EE	#FFF0F5	#FFEFDB
#FFEFD5	#FFEC8B	#FFEBCD	#FFE7BA
#FFE4E1	#FFE4C4	#FFE4B5	#FFE1FF
#FFDEAD	#FFDAB9	#FFD700	#FFD39B
#FFC1C1	#FFC125	#FFC0CB	#FFBBFF
#FFB90F	#FFB6C1	#FFB5C5	#FFAEB9
#FFA54F	#FFA500	#FFA07A	#FF8C69
#FF8C00	#FF83FA	#FF82AB	#FF8247
#FF7F50	#FF7F24	#FF7F00	#FF7256
#FF6EB4	#FF6A6A	#FF69B4	#FF6347
#FF4500	#FF4040	#FF3E96	#FF34B3
#FF3030	#FF1493	#FF00FF	#FF0000
#FDF5E6	#FCFCFC	#FAFAFA	#FAFAD2
#FAF0E6	#FAEBD7	#FA8072	#F8F8FF
#F7F7F7	#F5FFFA	#F5F5F5	#F5F5DC
#F5DEB3	#F4F4F4	#F4A460	#F2F2F2
#F0FFFF	#F0FFF0	#F0F8FF	#F0F0F0
#F0E68C	#F08080	#EEEEE0	#EEEED1
#EEEE00	#EEE9E9	#EEE9BF	#EEE8CD
#EEE8AA	#EEE685	#EEE5DE	#EEE0E5
#EEDFCC	#EEDC82	#EED8AE	#EED5D2
#EED5B7	#EED2EE	#EECFA1	#EECBAD
#EEC900	#EEC591	#EEB4B4	#EEB422
#EEAEEE	#EEAD0E	#EEA9B8	#EEA2AD
#EE9A49	#EE9A00	#EE9572	#EE82EE
#EE8262	#EE7AE9	#EE799F	#EE7942
#EE7621	#EE7600	#EE6AA7	#EE6A50

#EE6363	#EE5C42	#EE4000	#EE3B3B
#EE3A8C	#EE30A7	#EE2C2C	#EE1289
#EE00EE	#EE0000	#EDEDED	#EBEBEB
#EAEAEA	#E9967A	#E8E8E8	#E6E6FA
#E5E5E5	#E3E3E3	#E0FFFF	#E0EEEE
#E0EEE0	#E0E0E0	#E066FF	#DEDEDE
#DEB887	#DDA0DD	#DCDCDC	#DC143C
#DBDBDB	#DB7093	#DAA520	#DA70D6
#D9D9D9	#D8BFD8	#D6D6D6	#D4D4D4
#D3D3D3	#D2B48C	#D2691E	#D1EEEE
#D1D1D1	#D15FEE	#D02090	#CFCFCF
#CDCDC1	#CDCDB4	#CDCD00	#CDC9C9
#CDC9A5	#CDC8B1	#CDC673	#CDC5BF
#CDC1C5	#CDC0B0	#CDBE70	#CDBA96
#CDB7B5	#CDB79E	#CDB5CD	#CDB38B
#CDAF95	#CDAD00	#CDAA7D	#CD9B9B
#CD9B1D	#CD96CD	#CD950C	#CD919E
#CD8C95	#CD853F	#CD8500	#CD8162
#CD7054	#CD69C9	#CD6889	#CD6839
#CD661D	#CD6600	#CD6090	#CD5C5C
#CD5B45	#CD5555	#CD4F39	#CD3700
#CD3333	#CD3278	#CD2990	#CD2626
#CD1076	#CD00CD	#CD0000	#CCCCCC
#CAFF70	#CAE1FF	#C9C9C9	#C7C7C7
#C71585	#C6E2FF	#C67171	#C5C1AA
#C4C4C4	#C2C2C2	#C1FFC1	#C1CDCD
#C1CDC1	#C1C1C1	#C0FF3E	#BFEFFF
#BFBFBF	#BF3EFF	#BEBEBE	#BDBDBD
#BDB76B	#BCEE68	#BCD2EE	#BC8F8F

#BFFFFF	#BABABA	#BA55D3	#B9D3EE
#B8B8B8	#B8860B	#B7B7B7	#B5B5B5
#B4EEB4	#B4CDCD	#B452CD	#B3EE3A
#B3B3B3	#B2DFEE	#B23AEE	#B22222
#B0E2FF	#B0E0E6	#B0C4DE	#B0B0B0
#B03060	#AEEEEE	#ADFF2F	#ADD8E6
#ADADAD	#ABABAB	#AB82FF	#AAAAAA
#A9A9A9	#A8A8A8	#A6A6A6	#A52A2A
#A4D3EE	#A3A3A3	#A2CD5A	#A2B5CD
#A1A1A1	#A0522D	#A020F0	#9FB6CD
#9F79EE	#9E9E9E	#9C9C9C	#9BCD9B
#9B30FF	#9AFF9A	#9ACD32	#9AC0CD
#9A32CD	#999999	#9932CC	#98FB98
#98F5FF	#97FFFF	#96CDCD	#969696
#949494	#9400D3	#9370DB	#919191
#912CEE	#90EE90	#8FBC8F	#8F8F8F
#8EE5EE	#8E8E8E	#8E8E38	#8E388E
#8DEEEE	#8DB6CD	#8C8C8C	#8B8B83
#8B8B7A	#8B8B00	#8B8989	#8B8970
#8B8878	#8B8682	#8B864E	#8B8386
#8B8378	#8B814C	#8B7E66	#8B7D7B
#8B7D6B	#8B7B8B	#8B795E	#8B7765
#8B7500	#8B7355	#8B6969	#8B6914
#8B668B	#8B6508	#8B636C	#8B5F65
#8B5A2B	#8B5A00	#8B5742	#8B4C39
#8B4789	#8B475D	#8B4726	#8B4513
#8B4500	#8B3E2F	#8B3A62	#8B3A3A
#8B3626	#8B2500	#8B2323	#8B2252
#8B1C62	#8B1A1A	#8B0A50	#8B008B
#8B0000	#8A8A8A	#8A2BE2	#8968CD

#87CEFF	#87CEFA	#87CEEB	#878787
#858585	#848484	#8470FF	#838B8B
#838B83	#836FFF	#828282	#7FFFD4
#7FFF00	#7F7F7F	#7EC0EE	#7D9EC0
#7D7D7D	#7D26CD	#7CFC00	#7CCD7C
#7B68EE	#7AC5CD	#7A8B8B	#7A7A7A
#7A67EE	#7A378B	#79CDCD	#787878
#778899	#76EEC6	#76EE00	#757575
#737373	#71C671	#7171C6	#708090
#707070	#6E8B3D	#6E7B8B	#6E6E6E
#6CA6CD	#6C7B8B	#6B8E23	#6B6B6B
#6A5ACD	#698B69	#698B22	#696969
#6959CD	#68838B	#68228B	#66CDAA
#66CD00	#668B8B	#666666	#6495ED
#63B8FF	#636363	#616161	#607B8B
#5F9EA0	#5E5E5E	#5D478B	#5CACEE
#5C5C5C	#5B5B5B	#595959	#575757
#556B2F	#555555	#551A8B	#54FF9F
#548B54	#545454	#53868B	#528B8B
#525252	#515151	#4F94CD	#4F4F4F
#4EEE94	#4D4D4D	#4B0082	#4A708B
#4A4A4A	#48D1CC	#4876FF	#483D8B
#474747	#473C8B	#4682B4	#458B74
#458B00	#454545	#43CD80	#436EEE
#424242	#4169E1	#40E0D0	#404040
#3D3D3D	#3CB371	#3B3B3B	#3A5FCD
#388E8E	#383838	#36648B	#363636
#333333	#32CD32	#303030	#2F4F4F
#2E8B57	#2E2E2E	#2B2B2B	#292929
#282828	#27408B	#262626	#242424

#228B22	#218868	#212121	#20B2AA
#1F1F1F	#1E90FF	#1E1E1E	#1C86EE
#1C1C1C	#1A1A1A	#191970	#1874CD
#171717	#141414	#121212	#104E8B
#0F0F0F	#0D0D0D	#0A0A0A	#080808
#050505	#030303	#00FFFF	#00FF7F
#00FF00	#00FA9A	#00F5FF	#00EEEE
#00EE76	#00EE00	#00E5EE	#00CED1
#00CDCD	#00CD66	#00CD00	#00C5CD
#00BFFF	#00B2EE	#009ACD	#008B8B
#008B45	#008B00	#00868B	#00688B
#006400	#0000FF	#0000EE	#0000CD
#0000AA	#00008B	#000080	#000000

https://blog.csdn.net/liushengyulsy

3.read

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
功能：从fd所代表的文件中读取count个字节数据放到buf所指向的空间中
参数：
    fd:读取的那个文件的描述符
    buf:指针，指向的空间用来保存从文件中读取到的数据
    count:你想要读取多少个字节放到buf所指向的空间中
返回值：
    成功，返回实际读取到的字节数(<=count)
    返回0：已经读到文件的末尾EOF(end if file)
    失败：-1，error被设置
```

4.lseek

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int fd, off_t  offset,  int  whence);
功能：重新定位文件偏移量(光标)
参数
    fd:要进行重新定位光标的那个文件的fd
    offset:偏移量，数字，具体的含义要结合第三个参数
    whence：定位的方式，从哪个位置开始偏移
        SEEK_SET:文件开头
        SEEK_END:文件末尾
        SEEK_CUR:当前光标的位置
返回值
```

```
    成功：返回新光标距离文件开头的字节数
    失败：-1，error被设置
```

5.close

```
#include <unistd.h>

int close(int fd);
```

2.获取进程当前的工作目录

```
#include <unistd.h>

char *getcwd(char *buf, size_t size);
buf:指向的空间保存获取到的字符串
size:指向buf指向的空间，最多可以拷贝多少个字节
    目录字符串的长度 > size-1
    直接返回错误码，就不会出现数组越界的访问
返回值：
    成功：返回获取的目录字符串的地址
    失败：NULL，设置错误码

char *getwd(char *buf);
buf:指向的空间保存获取到的字符串
返回值：
    成功：返回获取的目录字符串的地址
    失败：NULL，设置错误码

char *get_current_dir_name(void);
```

3.改变当前进程的工作路径

```
#include <unistd.h>

int chdir(const char *path);
path:要改变的工作路径
返回值：
    成功 0
    失败 -1
int fchdir(int fd);
fd:需要使用open函数打开要改变的工作路径(打开目录应该需要只读权限)
返回值：
    成功 0
    失败 -1
```

练习：

```
改变工作路径为/home/china
接着去调用getcwd打印当前的工作路径
```

4.文件的截短

```
#include <unistd.h>
#include <sys/types.h>

int truncate(const char *path, off_t length);
path:文件名
length:截短的长度
    legth < 原来的长度
        截短
    legth > 原来的长度
        后面会给你补相应的字符(\0 nul)
        "扩展留下空洞"
返回值:
    成功 0
    失败 -1
int ftruncate(int fd, off_t length);
```

5.删除文件

```
#include <unistd.h>

int unlink(const char *pathname);
int rmdir(const char *pathname);

#include <stdio.h>

int remove(const char *pathname);
```

6.获取文件属性

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int stat(const char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
int lstat(const char *pathname, struct stat *statbuf);
pathname:带路径的文件名，用来指定要获取状态信息的哪个文件
statbuf:结构体指针，指向了一个struct stat类型的结构体空间，用来存放获取到的文件的属性信息的
fd:文件描述符，需要把文件用open打开
返回值:
    成功 0
    失败 -1 error被设置
```

```
struct stat {
    dev_t     st_dev;         /* ID of device containing file */
    //设备id
    ino_t     st_ino;         /* Inode number */
    //文件inode的结点编号
    mode_t    st_mode;        /* File type and mode */
```

```
    //文件的类型或权限，就可以用来判断文件是什么样的类型，具有哪种操作权限
    nlink_t   st_nlink;       /* Number of hard links */
    //硬链接数量
    uid_t     st_uid;         /* User ID of owner */
    //文件所有者的用户id
    gid_t     st_gid;         /* Group ID of owner */
    //文件所有者的组用户id
    dev_t     st_rdev;        /* Device ID (if special file) */
    //设备id
    off_t     st_size;        /* Total size, in bytes */
    //文件大小
    blksize_t st_blksize;     /* Block size for filesystem I/O */
    //块的大小(取决于文件系统) 512
    blkcnt_t  st_blocks;      /* Number of 512B blocks allocated */
    //块的数量

    /* Since Linux 2.6, the kernel supports nanosecond
                 precision for the following timestamp fields.
                 For the details before Linux 2.6, see NOTES. */

    struct timespec st_atim;  /* Time of last access */
    //最后一次访问的时间
    struct timespec st_mtim;  /* Time of last modification */
    //最后一次修改的时间
    struct timespec st_ctim;  /* Time of last status change */
    //最后一次修改文件状态的时间

    #define st_atime st_atim.tv_sec      /* Backward compatibility */
    #define st_mtime st_mtim.tv_sec
    #define st_ctime st_ctim.tv_sec
};
```

练习：判断一个文件的类型，文件名的名字从命令行传入

7.读取目录的内容

打开目录opendir

```
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name);
name:打开目录的路径
返回值：
    成功：返回一个DIR*指针，用来表示一个被打开的目录
    失败：NULL，errno被设置
DIR *fdopendir(int fd);
```

读取目录readdir

```
#include <dirent.h>

struct dirent *readdir(DIR *dirp);
dirp:一个打开的目录的指针
返回值:
    成功:指向下一个目录项的指针
    失败:没有更多的目录或出错 NULL
```

每一个目录项由一个结构体struct dirent来描述的，存储的就是一个个的struct dirent的结构体的数据

```
struct dirent {
    ino_t          d_ino;       /* Inode number */
    //目录文件inode编号
    off_t          d_off;       /* Not an offset; see below */
    //目录项的偏移量
    unsigned short d_reclen;    /* Length of this record */
    //目录项的大小
    unsigned char  d_type;      /* Type of file; not supported by all filesystem
types */
    //目录项类型
    char           d_name[256]; /* Null-terminated filename */
    //目录项的名字 不带任何路径
};
```

关闭目录closedir

```
#include <sys/types.h>
#include <dirent.h>

int closedir(DIR *dirp);
```

练习1：打印/home/china的子文件以及子目录的名字

练习2：计算给定目录下面，有多少普通文件和目录文件

练习3：把一个目录下面所有的文件名(普通文件或目录)都打印出来(打印路径:包括路径+名字)

练习4：假设一个目录的大小为：该目录下面及目录的目录下面...的所有普通文件的大小之和，写一个函数，求一个目录的大小(递归)

练习5：写一个双向循环链表，把一个目录下面所有的.c文件(包含其子目录下面的.c)的路径作为是一个结点的数据，最后打印出来这条链表